# Chapter 5

# Basic search strategies for solving CSPs

**Chapter 5. Basic search strategies for solving CSPs**

5.1 Introduction

5.2 General Search Strategies

5.3 Lookahead Strategies

5.4 Gather-information-while-searching Strategies

5.5 Hybrid Algorithms and Truth Maintenance

5.6 Comparison of Algorithms

## 5.1 Introduction

- **general search strategies**

Chronological backtracking strategy and *iterative broadening search*. Developed for general applications, do not use constraints propagation to improve the search.

- **lookahead strategies**

Following the commitment to a label, the problem is reduced through constraint propagation. Exploit that variables and domains are finite, and that constraints can be propagated.

- **gather-information-while-searching strategies**

To identify and record the sources of failure whenever backtracking is required during search. To gather and analyse information during the search. Avoids searching futile branches repeatedly. Exploits that sibling subtrees are very similar to each other in the search space.

For problems with $n$ variables, all solutions are located at depth $n$ of the search tree. This means that search strategies such as

- *breadth-first search,*
- *iterative deepening* (ID)
- *IDA**

cannot be very effective in CSP solving.

## 5.2 General Search Strategies

These strategies do not use the fact that constraints can be propagated in CSPs.

Because of the specific feature of search spaces, some general search strategies are more suitable than other.

### 5.2.1 Chronological backtracking

```
PROCEDURE Chronological_Backtracking( Z, D, C );
BEGIN
        BT-1( Z, { }, D, C );
END
```

```
PROCEDURE BT-1( UNLABELLED, COMPOUND_LABEL, D, C );
/* UNLABELLED is a set of variables to be labelled; */
/* COMPOUND_LABEL is a set of labels already committed to */
BEGIN
    IF (UNLABELLED = { }) THEN return(COMPOUND_LABEL)
    ELSE BEGIN
        Pick one variable x from UNLABELLED;
        REPEAT
            Pick one value v from D_x;
            Delete v from D_x;
            IF COMPOUND_LABEL + {<x,v>} violates no constraints
            THEN BEGIN
                Result ←
                    BT-1(UNLABELLED – {x}, COMPOUND_LABEL +
                        {<x,v>}, D, C);
                IF (Result ≠ NIL) THEN return(Result);
            END
        UNTIL (D_x = { });
        return(NIL); /* signifying no solution */
    END /* of ELSE */
END /* of BT-1*/
```

In BT, no attempt is made to use the constraints.

It is an exhaustive search which systematically explores the whole search space.

It is **complete** (all solutions could be found) and **sound** (all solutions found by it will satisfy all the constraints as required).

No attempt is made to prune off any part of the search space.

In the 8-queens problem, Chronological_Backtracking will search in the following way:

1 *A*
  2 *ABC*
    3 *ABCDE*
      4 *AB*
        5 *ABCD*
          6 *ABCDEFGH* (failed, and backtrack to 5)
        5 *EFGH* (start from A in row 6)
          6 *ABCDEFGH* (failed, and backtrack to 5)
        5 (failed, and backtrack to 4)
      4 *CDEFG*
        5 *AB* (start from A in row 5)
          6 *ABCD*
            7 *ABCDEF*
              8 *ABCDEFGH* (failed, and backtrack to 7)

### 5.2.2 Iterative Broadening

- The chronological backtracking algorithm exhausts one branch of the search tree before it turns to another when no solution is found.
- There is no reason to believe that earlier choices are more important than later ones.
- *Iterative Broadening* algorithm (IB) [1990] spreads the computational effort across the choices more evenly.
- The algorithm is basically the depth-first search with an artificial breadth cutoff threshold $b$.
- If a particular node has already been visited $b$ times (which include the first visit plus backtracking), then unvisited children will be ignored.
- If a solution is not found under the current threshold, then $b$ is increased.
- This process will terminate if a solution is found or (in order to ensure completeness) if $b$ is equal to or greater than the number of branches in all the nodes.

```
PROCEDURE IB-1( Z, D, C );
BEGIN
    b ← 1;
    REPEAT
        Result ← Breadth_bounded_dfs( Z, { }, D, C, b );
        b ← b + 1
    UNTIL ((b > maximum |D_x|) OR (Result ≠ NIL));
    IF (Result ≠ NIL) THEN return(Result);
    ELSE return( NIL );
END /* of IB-1 */
```

```
PROCEDURE Breadth_bounded_dfs( UNLABELLED, COM-
    POUND_LABEL, D, C, b );
/* depth first search with bounded breadth b */
BEGIN
    IF (UNLABELLED = { }) THEN return(COMPOUND_LABEL)
    ELSE BEGIN
            Pick one variable x from UNLABELLED;
            REPEAT
                Pick one value v from Dx;
                Delete v from Dx;
                IF (COMPOUND_LABEL + {<x,v>} violates no con-
                    straints)
                THEN BEGIN
                        Result ← Breadth_bounded_dfs(UN-
                            LABELLED – {x}, COMPOUND_LA-
                            BEL + {<x,v>}, D, C, b);
                        IF (Result ≠ NIL) THEN return(Result);
                    END
            UNTIL (Dx = { }) OR (b values in Dx have been tried);
            return(NIL);            /* signifying no solution */
        END /* ELSE */
END /* of Breadth_bounded_dfs */
```

- When the depth is large, IB could lead to computational speed-up over BT when there are enough solutions in the leaves of the search tree —when the total number of solutions at the leaves is greater than $e^{b/2}$, where $b$ is the branching factor of the search tree.

- Empirically, IB is tested on randomly generated problems. Results obtained agree with the theoretical analysis.

- Above analysis is based on the assumption that the search is totally uninformed (i.e. no heuristic exists).

- The performance of IB can be improved significantly when heuristics are available, even if the heuristic being used is very weak.

IB limitations as a general search method should be noted.

- Firstly, it is only useful for problems in which a single solution is required. If all solutions have to be found, IB will visit some solutions repeatedly and unnecessarily.

- Secondly, it has the same limitation as depth-first search, in that it can be trapped in branches with an infinite depth (in CSPs, all branches have a finite depth).

- Thirdly, the analysis is based on the assumption of a uniform branching factor throughout the search tree. In some problems, different subtrees may have different shapes (a different number of choice points and choices), and therefore the number of branches may vary under different subtrees.

If $A$ and $B$ are two sibling nodes which have $m$ and $n$ children, respectively, where $m$ is much greater than $n$, then subtrees under $B$ would be searched repeatedly (futilely) at the stage when $m < \text{Bound} \leq n$. IB need not be efficient in such problems.

Use of IB to CSPs where single solution is required:

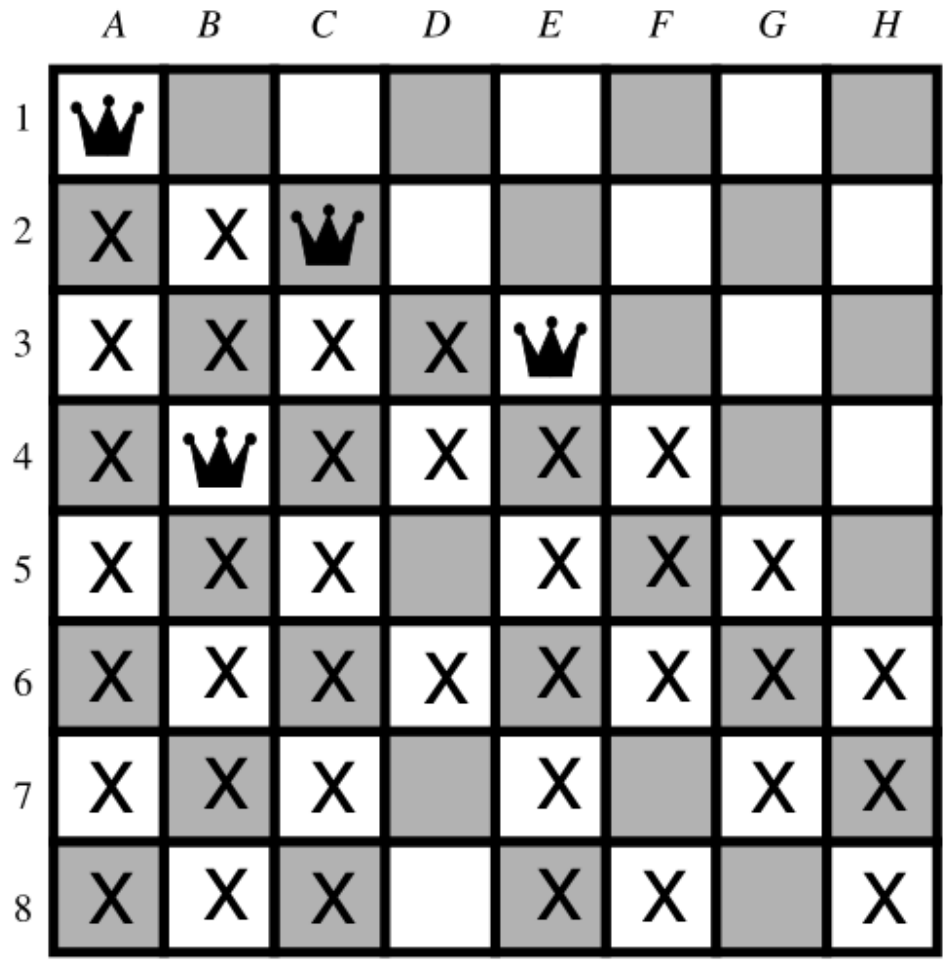
- in CSPs the depth of the search space is fixed;
- all subtrees can be made very similar to each other (by giving a fixed order to all the variables).

IB - modified when domain sizes of variables vary significantly.
To search a certain proportion of branches rather than a fixed number.
Pick the branches randomly, not in a given order.

The $N$-queens problem is a suitable application domain of IB because all variables have the same ($N$) possible values, and all subtrees have same depth.

## 5.3 Lookahead Strategies

Example:



As soon as column B is assigned to Queen 4:

- is possible to see that backtracking is needed without searching through all values for Queen 5
- no square in row 6 is safe, and therefore, it should not be necessary to go on with the current four committed queens.

When BT is used, columns D and H of Queen 5 will be tried before the program concludes that no position is available to put a queen in row 6.

The basic strategy for **lookahead** algorithms:

- to commit to one label at a time
- to reduce the problem at each step

### 5.3.1 Forward Checking

Forward Checking (FC) does exactly the same thing as BT, but

- it maintains the invariance that for every unlabelled variable *there exists at least one value* in its domain which is compatible with the labels that have been committed to.

Every time a label $L$ is committed to:

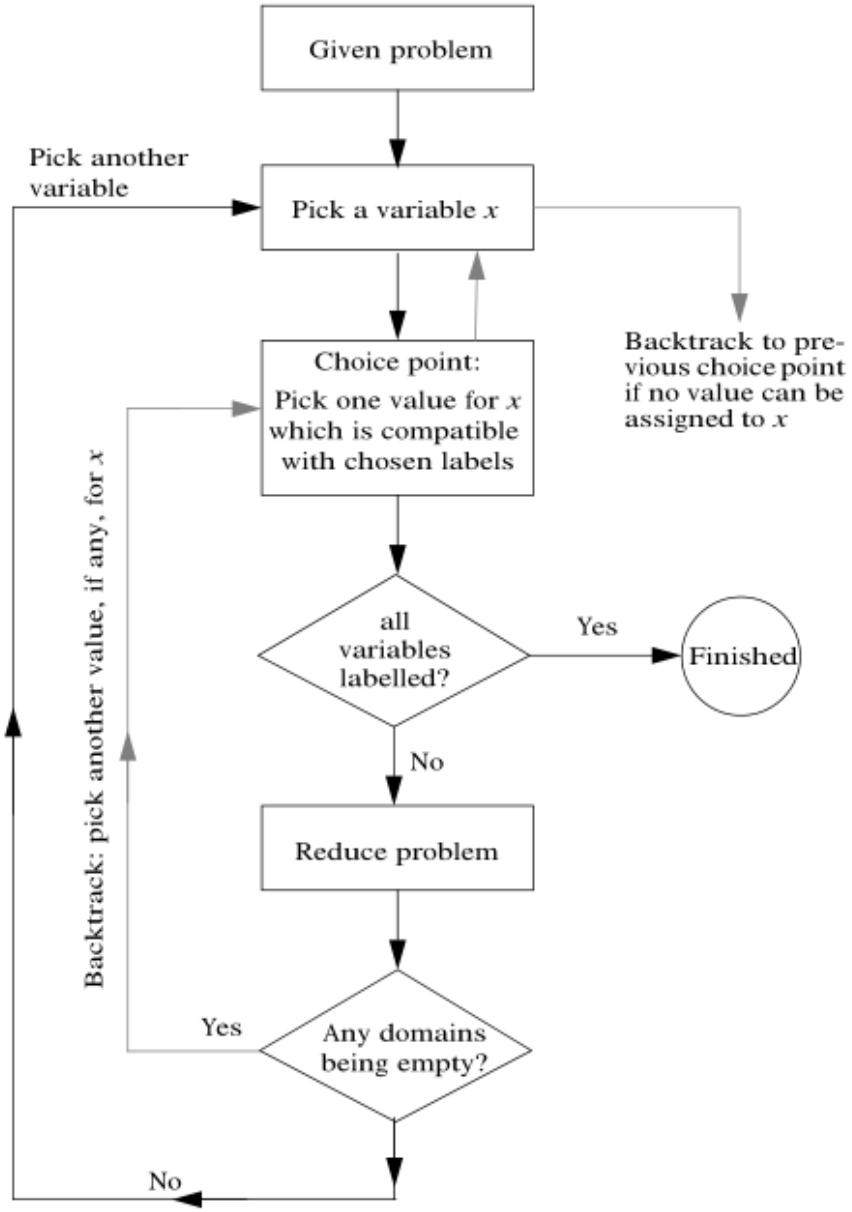
- FC will remove values from the domains of the unlabelled variables which are incompatible with $L$.

If the domain of any unlabelled variable is reduced to an empty set

- then $L$ will be rejected.
- otherwise, FC would try to label the unlabelled variable
- until all the variables have been labelled.

In case all the labels of the current variable have been rejected,

- backtrack to the previous variable as BT does.
- if there is no variable to backtrack to, the problem is insoluble.


Given problem
Pick another variable
Pick a variable $x$
Backtrack to previous choice point if no value can be assigned to $x$
Choice point:
Pick one value for $x$ which is compatible with chosen labels
Backtrack: pick another value, if any, for $x$
all variables labelled?
Yes
Finished
No
Reduce problem
Yes
Any domains being empty?
No

```
PROCEDURE Forward_Checking-1( Z, D, C );
BEGIN
        FC-1( Z, { }, D, C );
END /* of Forward_Checking-1 */
```

```
PROCEDURE FC-1( UNLABELLED, COMPOUND_LABEL, D, C );
BEGIN
    IF (UNLABELLED = { }) THEN return(COMPOUND_LABEL)
    ELSE BEGIN
        Pick one variable x from UNLABELLED;
        REPEAT
            Pick one value v from Dx; Delete v from Dx;
            IF (COMPOUND_LABEL + {<x,v>} violates no constraints)
            THEN BEGIN
                D' ← Update-1(UNLABELLED – {x}, D, C, <x,v>);
                IF (no domain in D' is empty)
                THEN BEGIN
                            Result ← FC-1(UNLABELLED – {x}, COM-
                                POUND_LABEL + {<x,v>}, D', C);
                            IF (Result ≠ NIL) THEN return(Result);
                        END;
            END
        UNTIL (Dx = { });
        return(NIL);            /* signifying no solution */
    END /* of ELSE */
END /* of FC-1 */
```

```
PROCEDURE Update-1(W, D, C, Label);
BEGIN          /* it considers binary constraints only */
    D' ← D;
    FOR each variable y in W DO;
        FOR each value v in D'y DO;
            IF (<y,v> is incompatible with Label with respect to the con-
                straints in C)
            THEN D'y ← D'y – {v};
    return(D');
END /* of Update-1 */
```

The main difference between FC-1 and BT-1:

- FC-1 calls Update-1 every time after a label is committed to
- this updates the domains of the unlabelled variables.
- if any domain is reduced to empty, FC-1 rejects the current label.

Update-1 considers binary constraints only.
FC-1 still needs to check the compatibility between the newly picked label and the committed labels.

One variation of FC-1:

- Maintain the invariance that for every unlabelled variable, there exists at least one value in its domain which is compatible with *all* the labels that have so far been committed to.
- In that case, no checking is required between any newly picked label and the compound label passed to FC-1.
- The revised procedures are called FC-2 and Update-2:

```
PROCEDURE FC-2( UNLABELLED, COMPOUND_LABEL, D, C );
BEGIN
    IF (UNLABELLED = { }) THEN return(COMPOUND_LABEL)
    ELSE BEGIN
        Pick one variable x from UNLABELLED;
        REPEAT
            Pick one value v from Dx; Delete v from Dx;
            D' ← Update-2(UNLABELLED – {x}, D, C, COMPOUND_-
                LABEL + {<x,v>});
            IF (no domain in D' is empty)
            THEN BEGIN
                        Result ← FC-2(UNLABELLED – {x}, COM-
                            POUND_LABEL + {<x,v>}, D', C);
                        IF (Result ≠ NIL) THEN return(Result);
                    END;
        UNTIL (Dx = { });
        return(NIL);            /* signifying no solution */
    END /* ELSE */
END /* of FC-2 */
```

```
PROCEDURE Update-2(W, D, C, COMPOUND_LABEL);
BEGIN /* it considers all constraints (not just binary constraints) */
    D' ← D;
    FOR each variable y in W DO;
        FOR each value v in D'y DO;
            IF (<y,v> is incompatible with COMPOUND_LABEL with
                respect to constraints on y + variables of COMPOUND_-
                LABEL)
            THEN D'y ← D'y – {v};
    return(D');
END /* of Update-2 */
```

The following is a trace of the decisions made by the forward checking algorithm in solving the 8-queens problem (variables are searched from Queen 1 to Queen 8, and values are ordered from *A* to *H*):

1 *A*
2 *C*
3 *E*
4 *BG*
5 *B*
6 *D*
5 *D*
4 *H*
5 *B*
6 *D*
7 *F*
6 (no more unrejected value)
5 *D*
4 (no more unrejected value)
3 *F*
.....

### 5.3.2 The Directional AC-Lookahead algorithm

In forward checking, the domains of the unlabelled variables are only checked against the committed labels. It is possible to reduce the problem further by maintaining directional arc-consistency (DAC, Definition 3-12) in each step after a label has been committed to.

```
PROCEDURE DAC-Lookahead-1( Z, D, C );
BEGIN
        Give Z an arbitrary ordering <;
        DAC-L-1( Z, { }, D, C, <);
END /* of DAC-Lookahead-1 */
```

```
PROCEDURE DAC-L-1( UNLABELLED, COMPOUND_LABEL, D,
    C, <);
BEGIN
    IF (UNLABELLED = { }) THEN return(COMPOUND_LABEL)
    ELSE BEGIN
        Pick one variable x from UNLABELLED;
        REPEAT
            Pick one value v from D_x; Delete v from D_x;
            IF (COMPOUND_LABEL + {<x,v>} violates no constraints)
            THEN BEGIN
                D' ← Update-1(UNLABELLED – {x}, D, C, <x,v>);
                (UNLABELLED – {x}, D", C) ← DAC-1(UNLABELLED –
                    {x}, D', C, <);
                IF (no domain in D" is empty)
                THEN BEGIN
                        Result ← DAC-L-1(UNLABELLED – {x}, COM-
                            POUND_LABEL + {<x,v>}, D", C, <);
                        IF (Result ≠ NIL) THEN return(Result);
                    END;
            END /* of THEN */
        UNTIL (D_x = { });
        return(NIL);            /* signifying no solution */
    END /* of ELSE */
END /* of DAC-L-1 */
```

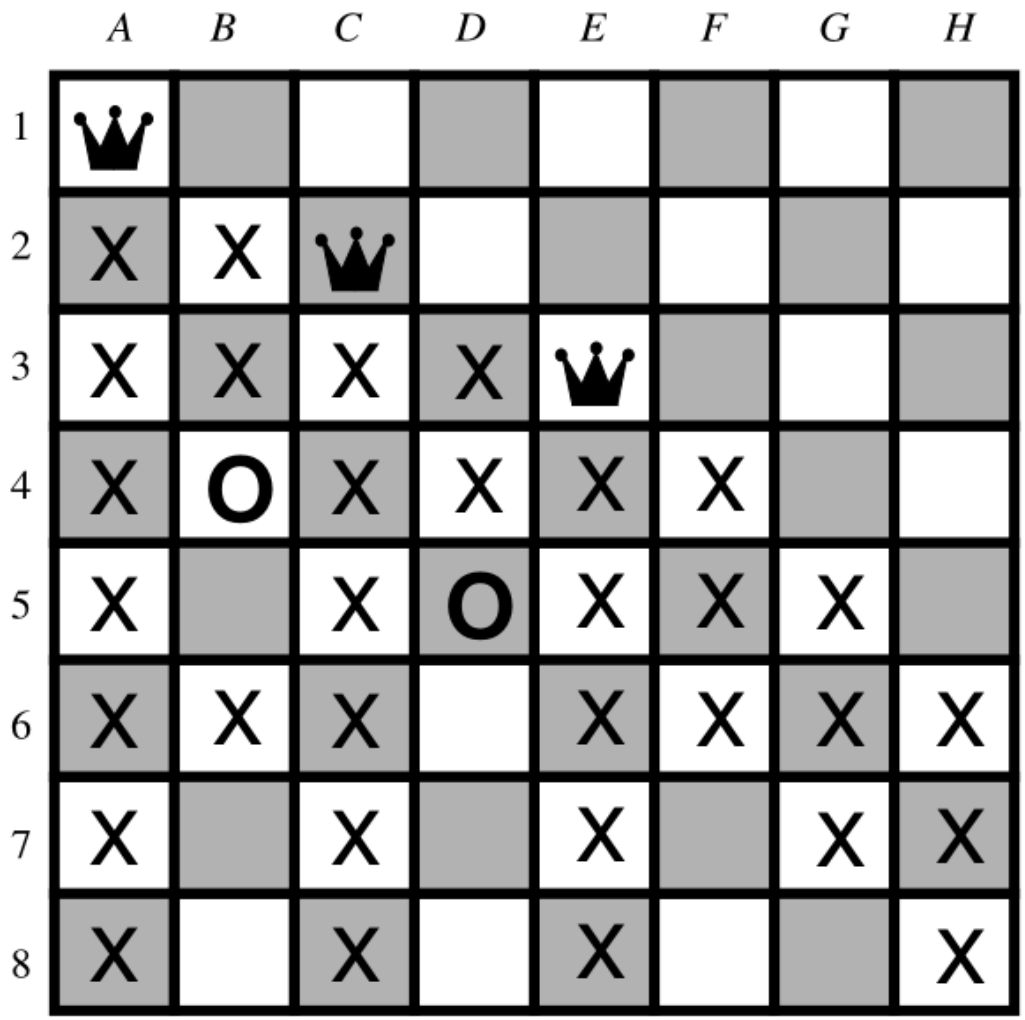



**Figure 5.3** Example showing the behaviour of DAC-Lookahead: <4,*B*> will be rejected at this stage because all the available values for Queen 6 are incompatible with it (squares marked **o** are rejected after DAC is achieved)

The following is a trace of applying the DAC-Lookahead algorithm to the 8-queens problem:

1 *A*
  2 *C*
    3 *E* (at this point, <4,*B*> and <5,*D*> are deleted)
      4 *GH*
        5 *B* (after maintaining DAC, no value for)
    3 *F* (at this point, <6,*D*> and <6,*E*> are deleted)
      4 *BH* (failed, and backtrack to 4)
    3 *G* (at this point, <5,*D*> and <7,*E*> are deleted)
      4 *B*
.....

Comparison Forward Checking vs DAC-Lookahead:

- a much smaller part of the search space is explored DAC-L
- maintenance costs for DAC among unlabelled variables

### 5.3.3 The AC-Lookahead algorithm

AC is a stronger property than DAC.
AC-lookahead (AC-L) maintains AC after committing to each label.
It is also referred to as *Full Lookahead* in the literature.

When a variable is labelled

- We remove from the domains of all unlabelled variables those values which are incompatible with the committed labels
- We maintain arc-consistency among the unlabelled variables.
- We ensure that there exists a pair of compatible labels between every pair of unlabelled variables.

```
PROCEDURE AC-Lookahead-1( Z, D, C );
BEGIN
        AC-L-1( Z, { }, D, C);
END /* of AC-Lookahead-1 */
```

```
PROCEDURE AC-L-1( UNLABELLED, COMPOUND_LABEL, D, C );
BEGIN
    IF (UNLABELLED = { }) THEN return(COMPOUND_LABEL)
    ELSE BEGIN
        Pick one variable x from UNLABELLED;
        REPEAT
            Pick one value v from D_x; Delete v from D_x;
            IF (COMPOUND_LABEL + {<x,v>} violates no constraints)
            THEN BEGIN
                D' ← Update-1(UNLABELLED – {x}, D, C, <x,v>);
                (UNLABELLED – {x}, D", C) ← AC-X(UNLABELLED –
                    {x}, D', C);
                IF (no domain in D" is empty)
                THEN BEGIN
                        Result ← AC-L-1(UNLABELLED-{x}, COM-
                            POUND_LABEL + {<x,v>}, D", C);
                        IF (Result ≠ NIL) THEN return(Result);
                    END;
            END /* of THEN */
        UNTIL (D_x = { });
        return(NIL);            /* signifying no solution */
    END /* of ELSE */
END /* of AC-L-1 */
```

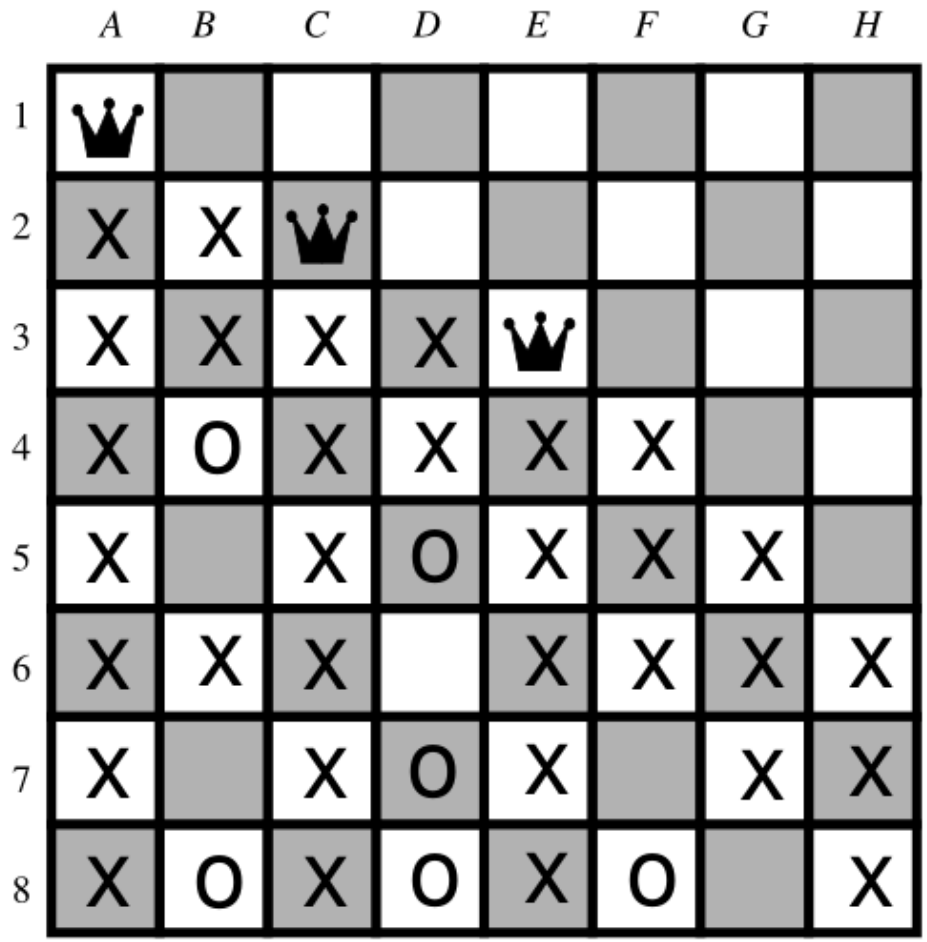



**Figure 5.4** Example showing the behaviour of AC-Lookahead: label <3,*E*> will be rejected if AC is to be achieved. Labels marked **O** are rejected for having no compatible values in Queen 6; then label <5, *H*> is rejected for having no compatible value for Queen 4; then <7,*B*> will be removed for having no compatible value with Queen 5; and finally, <7,*F*> conflicts with <8,*G*>, which are the only values left for Queens 7 and 8, respectively

Comparison DAC-Lookahead vs AC-Lookahead:

- AC-L spends more effort in problem reduction
- removes more redundant values from the domains of the unlabelled variables
- prunes off a larger part of the search space
- detects dead-ends at earlier stages

The following is a trace of an arc-consistency lookahead algorithm in the 8-queens problem:

1 *A*
  2 *C*
    3 *E*
    3 *F*
    3 *G*
    3 *H*
  2 *D*
    3 *B*
    3 *F*
.....

The DAC-Lookahead algorithm is called *partial lookahead* and the AC-Loohahead algorithm is called *full looking forward* in the literature [HarEll80]. The name *full looking forward* is quite misleading, as AC is not the limit of what one can achieve in problem reduction.

It is possible for algorithms to look further ahead by maintaining properties stronger than AC, such as PC or $k$-consistency for $k>2$.
In fact, if one maintains strong $k$-consistency when there are only $k$ unlabelled variables left, and no domain is left empty after doing so, then one can guarantee that no backtracking is needed in the search.

Whether it is justifiable to spend the effort to do so is another matter, and the decision probably depends on the application.
The more the variables constrain each other, and the tighter the constraints, the more return one can get by maintaining a greater level of consistency.

## 5.4 Gather-information-while-searching Strategies

The fact that sibling subtrees in the search space are similar in the search trees of CSPs allows one to learn from experience in a search. When backtracking is required, one **could analyse the reason for the failure**, so as to avoid making the same mistake repeatedly in the future.

### 5.4.1 Dependency directed backtracking

DDBT is a general strategy that can be applied to various problems. Identify the culprit(s) when failures occur so the algorithm can backtrack to relevant decisions only.
In CSPs, the culprit(s) may be identifiable using the constraints in the problem. Therefore, it is possible to apply DDBT to CSPs efficiently.

#### 5.4.1.1 BackJumping

When BJ needs to backtrack, identifies the culprit decisions (commitments to labels) which have jointly caused the failure.

If every value in the domain of the current variable is in conflict with some committed labels, then BJ backtracks to the most recent culprit decision rather than the immediate past variable as is the case in BT.

In CSPs, the culprits can be identified by enumerating the values in the current variable, and using the constraints as guidance to find out why they are rejected.

If the current variable was labelled and then backtracked to, then BJ will backtrack to the immediate past variable. This is because at least one value in the domain of the current variable has passed all the compatibility checking with the labels committed to so far.

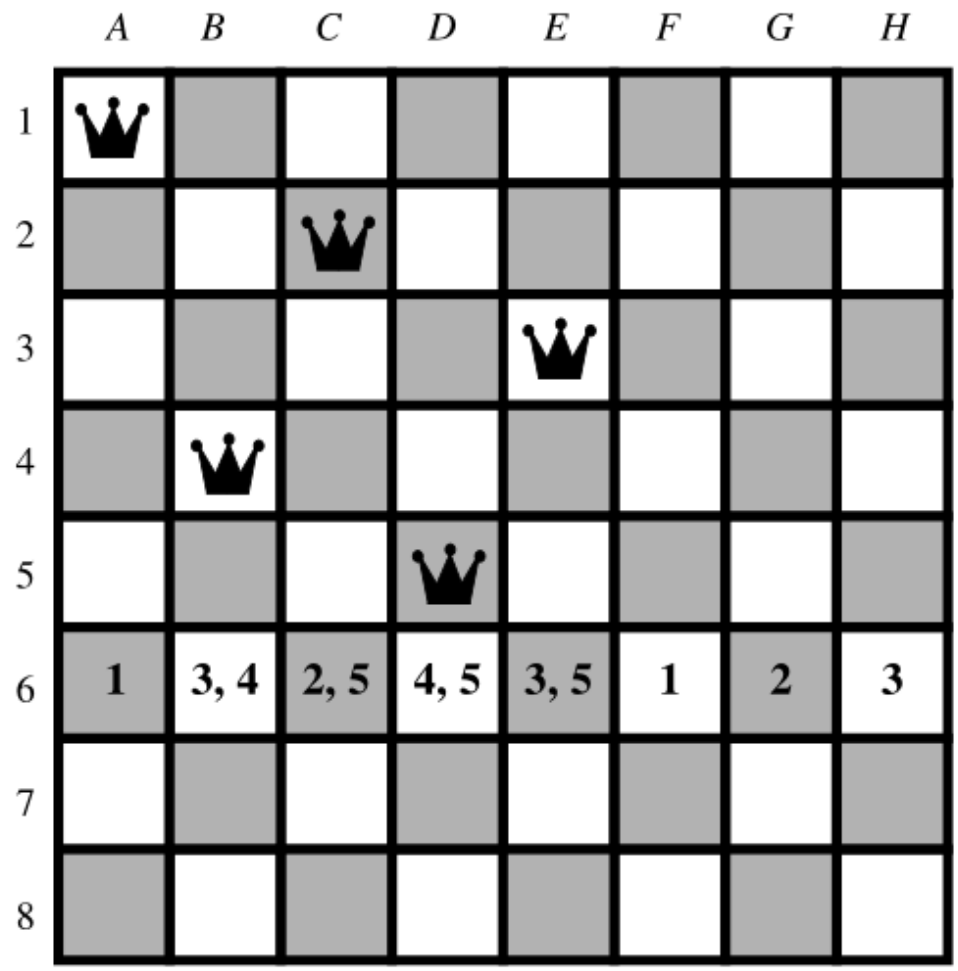



**Figure 5.5** A board situation in the 8-queens problem. The numbers in row 6 indicate the labelled queens that the corresponding squares are incompatible with. It is possible at this stage to realize that changing the value of Queen 5 will not resolve the conflicts, or learn incompatible compound labels

```
PROCEDURE BackJumping( Z, D, C );
BEGIN
        BJ-1( Z, { }, D, C, 1 );
END /* of BackJumping */
```

```
PROCEDURE BJ-1( UNLABELLED, COMPOUND_LABEL, D, C, L );
/* Let Level_of be a global array of integers, one per variable, for
    recording levels; BJ-1 either returns a solution tuple or a Level to
    be backtracked to */
BEGIN
    IF (UNLABELLED = { }) THEN return(COMPOUND_LABEL)
    ELSE BEGIN
            Pick one variable x from UNLABELLED; Level_of[x] ← L;
            TD_x ← D_x;  /* TD is a copy of D_x, used as working storage */
            REPEAT
                v ← any value from TD_x; TD_x ← TD_x – {x};
                IF (COMPOUND_LABEL + {<x,v>} violates no con-
                    straints)
                THEN BEGIN
                            Result ← BJ-1(UNLABELLED – {x}, COM-
                                POUND_LABEL + {<x,v>}, D, C, L + 1 );
                            IF (Result ≠ backtrack_to(Level))      /* back-
                                track_to(Level) is a data structure signify-
                                ing backtracking and where to */
                            THEN return(Result);
                        END /* of IF within the REPEAT loop */
            UNTIL ((TD_x = { }) OR (Result = backtrack_to(Level) AND
                Level < L));
            IF (Result = backtrack_to(Level) AND Level < L)
            THEN return(backtrack_to(Level));
            ELSE BEGIN     /* all the values in D_x have been rejected */
                        Level ← Analyse_bt_level( x, COMPOUND_LA-
                            BEL, D_x, C, L );
                        return(backtrack_to(Level));
                                    /* return a special data structure */
                    END
        END /* of ELSE */
END /* of BJ-1 */
```

```
PROCEDURE Analyse_bt_level( x, Compound_label, D_x, C, L );
BEGIN
    Level ← −1;
    FOR each (a ∈ D_x) DO
        BEGIN
            Temp ← L − 1; NoConflict ← True;
            FOR each <y,b> ∈ Compound_label DO
                    IF NOT satisfies((<x,a><y,b>), C_x,y)
                    THEN BEGIN
                                Temp ← Min( Temp, Level_of[y] );
                                NoConflict ← False;
                            END
            IF (NoConflict) THEN return(Level_of[x] − 1)
                                ELSE Level ← Max( Level, Temp );
        END
    return( Level );
END /* of Analyse_bt_level */
```

Here we show part of the search space that BJ explores:

1 *A*
- 2 *ABC*
  - 3 *ABCDE*
    - 4 *AB*
      - 5 *ABCD*
        - 6 *ABCDEFGH* (failed, backtrack to Queen 4)
      - 5 (skipped, as it is an irrelevant decision)
    - 4 *CDEFG*
      - 5 *AB*
        - 6 *ABCD*
          - 7 *ABCDEF*
            - 8 *ABCDEFGH* (failed, backtrack to Queen 6)
          - 7 (skipped, as it is an irrelevant decisions)
        - 6 *EFGH* (failed, backtrack to Queen 5)

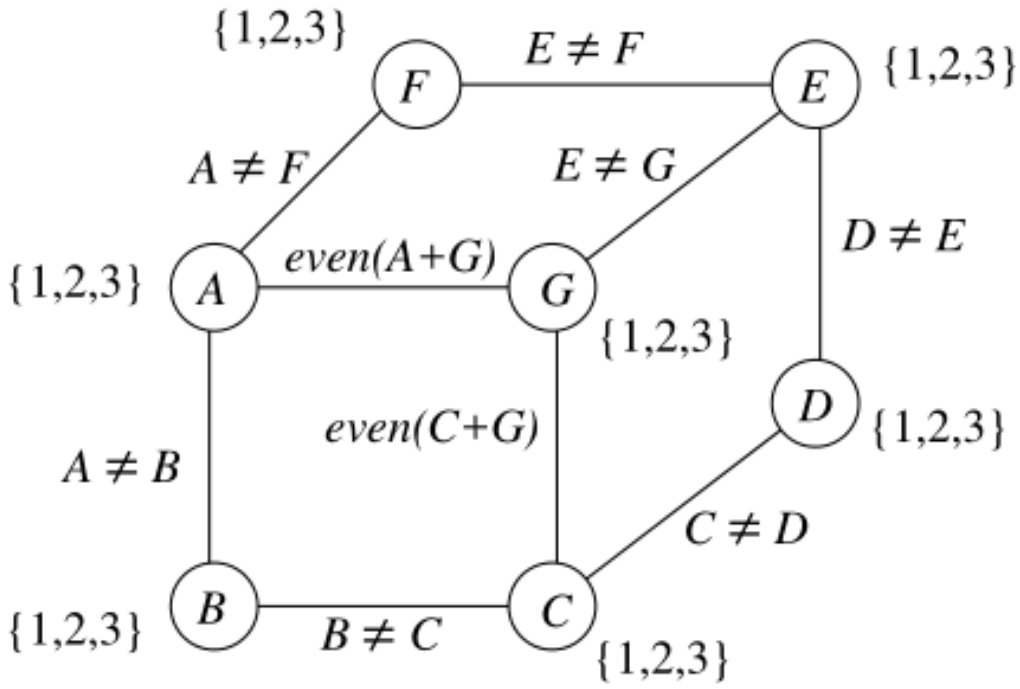
.....

#### 5.4.1.3 Graph-based BackJumping

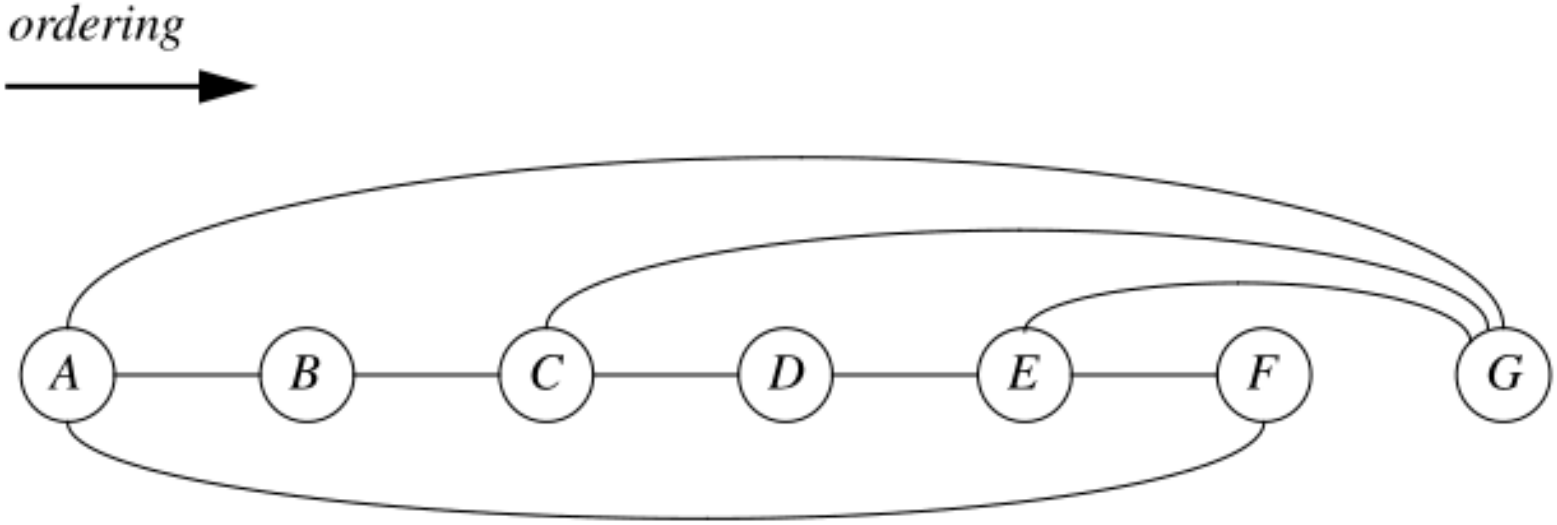
BJ jumps back to the most recent culprit decision rather than the previous variable.
GBJ backtracks to the most recent variable that constrains the current variable if all the values for the current variable are in conflict with some committed labels. Like BJ, GBJ will backtrack to the last variable when the current variable has been labelled and backtracked to (in which case, at least one value has passed all the compatibility checking).

By making use of the constraints, GBJ manages to jump past the decisions which are definitely irrelevant to the failure. Using the constraints saves computational effort in the analysis. GBJ is not guaranteed to identify the real culprit at all times.

**Figure 5.6** An example CSP for illustrating the GBJ algorithm (the nodes in the constraint graph represent the variables, and the edges represent the constraints)




(a) A constraint graph: variables are $A$, $B$, $C$, $D$, $E$, $F$ and $G$; domains for all the variables are {1,2,3}; constraints are shown on the edges




(b) Ordering of the variables

### 5.4.2 Learning nogood compound labels algorithms

DDBT analyses failure situations in order to identify culprit decisions which have jointly caused the current variable to be over-constrained.

The only piece of information it uses after the analysis is the level to backtrack to.

We can retain the information about combinations of labels which caused the problem to be over-constrained. We refer to one such combination as *nogood sets*.

**Definition 5-1:**
A set $S$ is a **covering set** of a set of sets $SS$ if and only if for every set $S'$ in $SS$, there exists at least one element in $S'$ which is in $S$. In this case, we say that $S$ **covers** $SS$:

$$\text{covering_set}(S, SS) \equiv (\forall\, S' \in SS: (\exists\, m \in S': m \in S)) \blacksquare$$

**Definition 5-2:**
A set $S$ is a **minimal covering set** of a set of sets $SS$ if and only if $S$ is a covering set of $SS$, and no subset of $S$ is a covering set of $SS$.

$$\text{minimal_covering_set}(S, SS) \equiv$$
$$(\text{covering_set}(S, SS) \wedge (\forall\, S' \subseteq S: \neg\, \text{covering_set}(S', SS))) \blacksquare$$

**Example:**
$SS$ be $\{\{a, b, c\}, \{a, d\}, \{b, c, d\}, \{d, e, f\}\}$. The sets $\{a, c, e\}$, $\{a, b, d\}$, $\{a, d\}$ and $\{b, d\}$ are all covering sets of $SS$. By definition, if an empty set is present in the set of sets $SS$, then $SS$ has no covering set. Here, $\{a, d\}$, $\{b, d\}$ and $\{a, c, e\}$ are minimal covering sets of $SS$.

```
PROCEDURE Learn_Nogood_Compound_Labels( Z, D, C );
BEGIN
        LNCL-1( Z, { }, D, C );
END
```

```
PROCEDURE LNCL-1( UNLABELLED, Compound_label, D, C );
BEGIN
    IF (UNLABELLED = { }) THEN return(Compound_label);
    ELSE BEGIN
        Pick one variable x from UNLABELLED;
        TD_x ← D_x;              /* TD is a copy of D_x, used as a working
            variable */
        REPEAT
            v ← any value from TD_x; TD_x ← TD_x – {x};
            IF ((Compound_label + {<x,v>} violates no constraints)
                AND (NOT Known_to_be_Nogood(Compound_label +
                {<x,v>})))
            THEN BEGIN
                        Result ← LNCL-1(UNLABELLED – {x}, Com-
                            pound_label + {<x,v>}, D, C);
                        IF (Result ≠ NIL) THEN return(Result);
                    END;
        UNTIL ( TD_x = { } OR Known_to_be_Nogood(Compound_la-
            bel) );
        IF (TD_x = { }) THEN Analyse_nogood( x, Compound_label, D_x,
            C );
        return(NIL);
    END /* of ELSE */
END /* of LNCL-1 */
```

```
PROCEDURE Analyse_nogood( x, Compound_label, D_x, C );
BEGIN
    /* identify conflict sets for each value */
    FOR each (a ∈ D_x) DO
        BEGIN
            Conflict_set[a] ← { };
            FOR each <y,b> ∈ Compound_label DO
                IF (NOT satisfies( (<x,a><y,b>), C_x,y ))
                THEN Conflict_set[a] ← Conflict_set[a] + <y,b>;
            IF (Conflict_set[a] = { }) THEN return;      /* no Nogood set
                identified */
        END
    /* identify Nogood sets */
    FOR each set of labels LS covering the cartesian set
        ∪_∀m Conflictset[m] DO
            IF NOT Known_to_be_Nogood(LS) THEN record nogood_-
                set(LS);
END /* of Analyse_nogood */
```

```
PROCEDURE Known_to_be_Nogood( CL );
BEGIN
    IF (there exists a compound label CL' such that (nogood_set(CL')
        is recorded) AND (CL' is subset of CL))
    THEN return( True )
    ELSE return( False );
END /* of Known_to_be_Nogood */
```

The LNCL-1 procedure is similar to BT-1 and BJ-1, except that when it needs to backtrack, the Analyse_nogood procedure is called.

The REPEAT loop in LNCL-1 terminates if all the values are exhausted, or if it has been learned that the current Compound_label is nogood.

LNCL identifies the minimal covering sets (which are subsets of all the Queens {1, 2, 3, 4, 5}) that has caused the failure of Queen 6 and remember them as conflict sets.

We are trying to identify all minimum covering sets of the set:

{{1}, {3, 4}, {2, 5}, {4, 5}, {3, 5}, {1}, {2}, {3}}.

- The elements of the singleton sets must participate in all covering sets, i.e. {1, 2, 3} must be included in all covering sets.
- Only the set {4, 5} is not covered by {1, 2, 3}.
- Just one of Queen 4 or Queen 5 are required.

{<1,*A*><2,*C*><3,*E*><4,*B*>} and {<1,*A*><2,*C*><3,*E*><5,*D*>} are nogood sets and will be recorded accordingly.

Since {<1,*A*><2,*C*><3,*E*><4,*B*>} is a conflict set, all other values of Queen 5 will not be tried (i.e. <5,*E*> need not be considered).

No matter what value one assigns to Queen 4 later, the combination <1,*A*>, <2,*C*>, <3,*E*> and <5,*D*> will never be tried.

Finding minimum covering sets is sometimes referred to as *deep-learning*. We could settle for non-minimal covering sets, called *shallow-learning*.

In general, the deeper one learns (i.e. the smaller covering sets one identifies), the more computation is required. In return, deep-learning allows one to prune off more search space than shallow-learning.

### 5.4.3 Backchecking and Backmarking

Both backchecking (BC) and its descendent backmarking (BM) are useful algorithms for reducing the number of compatibility checks.

#### 5.4.3.1 Backchecking

BC attempts to reduce the number of compatibility checks as much as possible. Applications where compatibility checks are computationally expensive.

When considering a label <*y*,*b*>, BC checks whether it is compatible with all the labels committed to so far. If <*y*,*b*> is found to be incompatible with the label <*x*,*a*>, then BC will remember this. As long as <*x*,*a*> is still committed to, <*y*,*b*> will not be considered again.

Values which are incompatible with the committed labels are rejected from the domains of the variables.

The difference to FC is that if <$x$,$a$> and <$y$,$b$> are incompatible with each other, and $x$ is labelled before $y$, then:

- FC will remove $b$ from $y$'s domain when $x$ is being labelled,
- BC will remove $b$ from $y$'s domain when $y$ is being labelled.

BC will not be able to backtrack as soon as FC, which anticipates failures.

BC is inferior to FC.

### 5.4.3.2 Backmarking

For each variable, BM records the highest level that it last backtracked to. This helps BM to avoid repeating those compatibility checks which are known to succeed or fail.

The key is to perform compatibility checks according to the chronological order of the variables — the earlier a label is committed to, the earlier it is checked against the currently considered label.

The algorithm BM for binary problems is illustrated below.

The following four sets of global variables are being used in Backmark-1.

(1) *Assignments*
For each variable $x$, *Assignment*($x$) records the value assigned to $x$.

(2) *Mark*s
For each label <$x$,v>, *Mark*($x$,$v$) records the shallowest variable (according to the chronological ordering of the variables) in which assignment is incompatible with <$x$,$v$>.

(3) *Ordering*
Variables are ordered, and if $x$ is the $k$-th variable, then *Ordering*($x$) = $k$.

(4) *LowUnit*s
For each variable $x$, *LowUnit*($x$) records the ordering of the shallowest variable $y$ which has had its *Assignment* changed since $x$ was last visited.

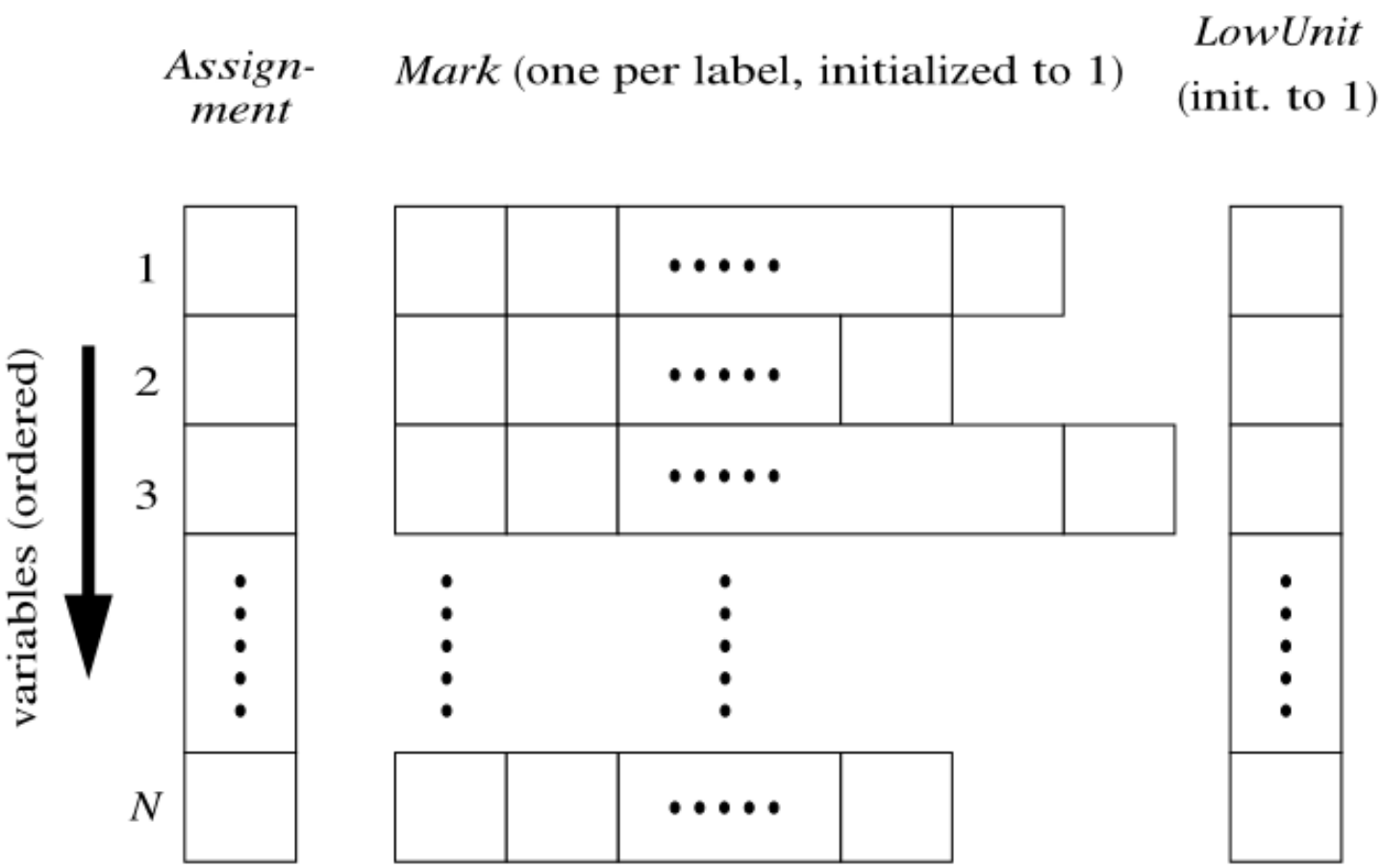



**Figure 5.7** Variable sets used by Backmark-1. *Assignment*(x) = the value assigned to variable *x*. *Mark*(*x*, *v*) = the lowest level at which <*x*, *v*> failed. *LowUnit*(x) = the ordering of the lowest variable *y* which *Assignment* has been changed since the last time *x* is visited

```
PROCEDURE Backmark-1( Z, D, C );
BEGIN
    i ← 1;
    For each x in Z DO
        BEGIN
            LowUnit(x) ← 1;
            FOR each v in D_x DO
                Mark(x,v) ← 1;
            Ordering(x) ← i; i ← i + 1;
        END /* end of initialization */
    BM-1( Z, { }, D, C, 1 );
END /* of Backmark-1 */
```

```
PROCEDURE BM-1(UNLABELLED, COMPOUND_LABEL, D, C,
    Level);
BEGIN
    IF (UNLABELLED = { }) THEN return(COMPOUND_LABEL)
    ELSE BEGIN
        x ← the variable in UNLABELLED which Ordering equals
            Level;
        Result ← NIL;
        REPEAT
            v ← any value from Dx; Dx ← Dx – {v};
            IF Mark(x,v) ≥ LowUnit(x)                    /* else reject v */
            THEN BEGIN
                        IF Compatible( <x,v>, COMPOUND_LABEL )
                        THEN Result ← BM-1(UNLABELLED – {x}, COM-
                            POUND_LABEL + {<x,v>}, D, C, Level + 1);
                    END
        UNTIL ((Dx = { }) OR (Result ≠ NIL));
```

```
        IF (Result = NIL) THEN
            /* all values tried and failed, so backtracking is required */
            BEGIN
                LowUnit(x) ← Level – 1;
                FOR each y in UNLABELLED DO
                    LowUnit(y) ← Min( LowUnit(y), Level – 1 )
            END
        return(Result);
    END
END /* of BM-1 */
```

```
PROCEDURE Compatible( <x,v>, COMPOUND_LABEL );
/* variables in COMPOUND_LABEL are ordered according to the glo-
    bal variable Ordering */
BEGIN
    Compatible ← True; y ← LowUnit(x);
    WHILE ((Ordering(y) < Ordering(x)) AND Compatible) DO
        BEGIN
            <y,v'> ← projection of COMPOUND_LABEL to y;
            IF satisfies( (<x,v><y,v'>), C_{x,y} )
            THEN y ← successor of y according to the Ordering;
            ELSE Compatible ← False;
        END;
    Mark(x,v) ← ordering(y); /* update global variable */
    return( Compatible );
END /* of Compatible */
```

BM-1 differs from BT-1 where compatibility is checked (the Compatible procedure) and in backtracking. In BM-1, *Ordering*($x$) is the order the variable $x$ is labelled. Values of the *Marks* are only changed in the Compatible procedure, and values of the *LowUnits* are only changed when backtracking takes place.

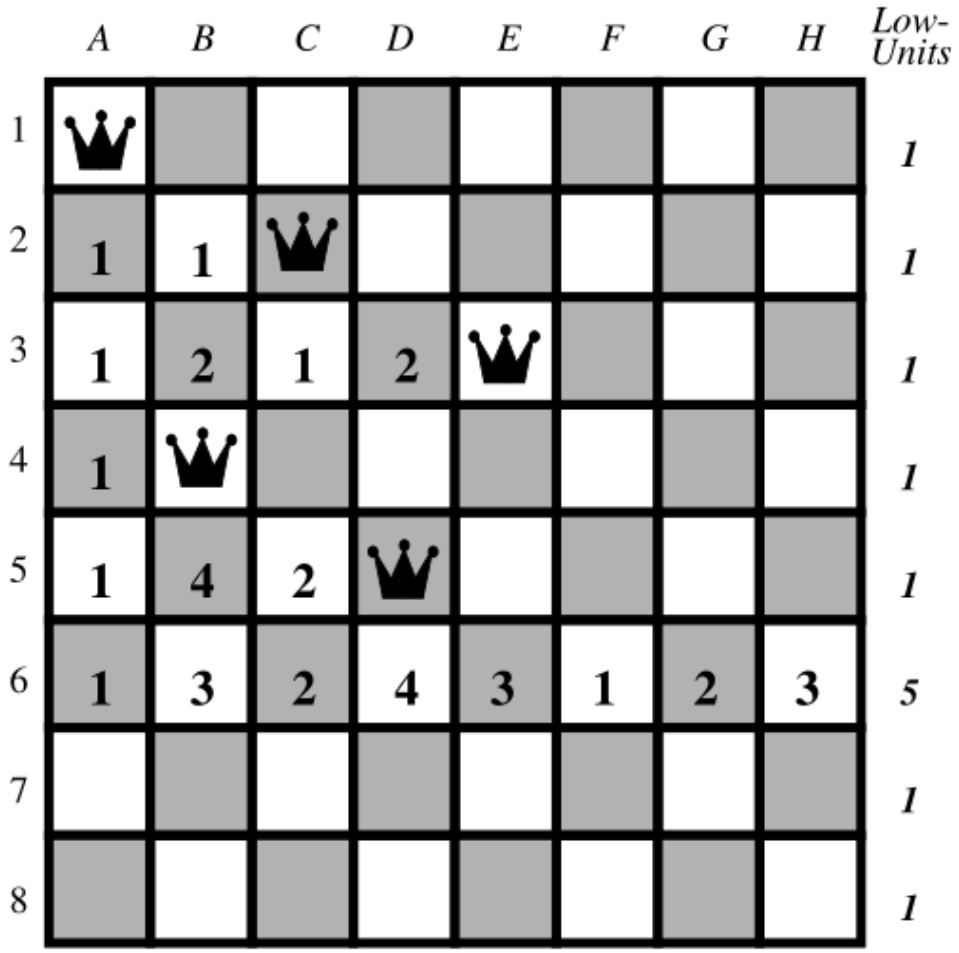



**Figure 5.8** A board situation in the 8-queens problem showing the values of *Marks* and *LowUnits* at some state during Backmarking. The number in each square shows the value of *Mark* for that square. At this state, all the values of Queen 6 have been rejected, and consequently, *LowUnit*)6) has been changed to 5. If and when all the values of Queen 5 are rejected, both *LowUnit*(5) and *LowUnit*(6) will be changed to 4.

## 5.5 Hybrid Algorithms and Truth Maintenance

It is possible to **combine** the lookahead strategies and the gather-information-whilesearching strategies, although doing so may not always be straightforward.

We illustrate the possible difficulties by considering the amalgamation of DAC-L (Section 5.3.2) and BJ (Section 5.4.1).

The BJ algorithm is able to compute the culprits because, by simply examining the constraints, it can identify the decisions which have caused each of the values to be rejected.

In DAC-L, values are not only rejected because they are incompatible with the current label, but also through problem reduction. Therefore, the culprits may not be identifiable through simple examination of the constraints.

Recognizing the cause of rejection is not at all an easy task.

The more inferences one performs to reduce a problem, the more difficult it is to identify the culprits in an *ad hoc* way.

One general solution to this problem is to attach the justifications to each inference made.

Attaching justifications or assumptions to the inferences is required by many AI researches. This research is known as *truth maintenance,*

General purpose systems for this task are called *Truth Maintenance Systems* (TMS).

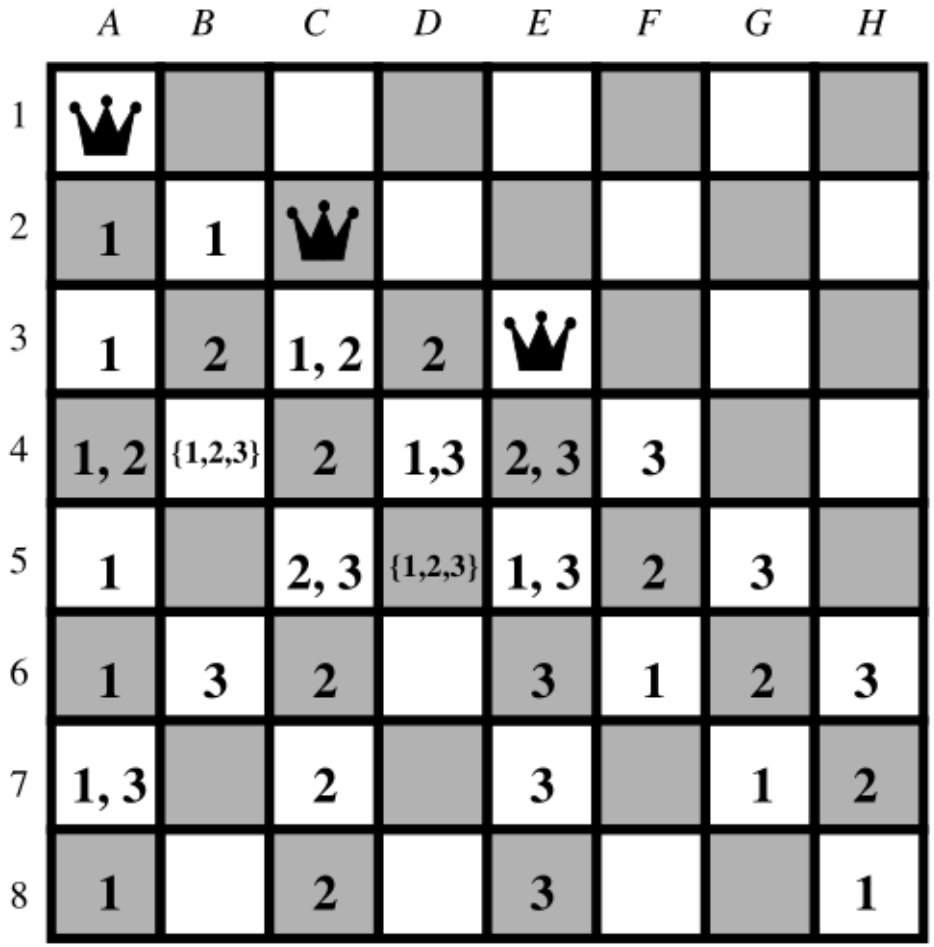



**Figure 5.9** A board situation in the 8-queens problem for showing the role of Truth Maintenance in a DAC-L and DDBT hybrid algorithm. The number(s) in each square shows justification(s) for rejecting that square; e.g. the number 1 means the subject square is rejected because it is incompatible with the committed Queen 1. Labels <4,*B*> and <5,*D*> are rejected because of the simultaneous commitments to the positions for Queens 1, 2 and 3 shown.

## 5.6 Comparison of Algorithms

- Comparing the efficiency of the above algorithms is far from straightforward.
- In terms of complexity, all the above algorithms have worst case complexity which is exponential to the number of variables.
- Empirically, one needs to test them over a large variety of problems in order to generalize any comparative results.
- There are many dimensions in which an algorithm's efficiency can be measured.
- For a particular application, the most important efficiency measure is probably run time. Unfortunately, the run time of a program could be affected by many factors, including the choice of programming language, the data structure, programming style, the machine used, etc.
- Some algorithms could be implemented more efficiently in one language than in another, and comparing the run time of programs written in different languages is not very significant in general.

In the literature, the following aspects (in addition to run time) have been used to compare the efficiency of different CSP solving algorithms:

- the number of nodes expanded in the search tree;
- the number of compatibility checks performed (sometimes referred to as *consistency checks* in the literature);
- the number of backtracking required.

Lookahead algorithms attempt to prune off search spaces, and therefore tend to expand less nodes. The cost of doing so is to perform more compatibility checks at the earlier stages of the search.

DDBT and learning algorithms attempt to prune off search spaces by jumping back to the culprit decisions and recording redundant compound labels. Their overhead involves the analysis of failures. These algorithms tend to expand less nodes in the search space and require less backtracking than BT.

BC and BM tend to require fewer compatibility checks than BT and the above algorithms, at the cost of overhead in maintaining certain records. Whether the overhead of all these algorithms is justifiable or not is very much problem-dependent.

One of the better known empirical comparisons Haralick & Elliott [1980].
Problems with up to 10 variables were tested, and in a problem with $N$ variables, each domain contains $N$ values.

Lookahead algorithms in general explores fewer nodes in the search space than the Backtracking, BC and BM algorithms.

- Among the lookahead algorithms, AC-Lookahead explores fewer nodes than DAC-Lookahead, which explores fewer than Forward Checking.
- Lookahead algorithms do more compatibility checking than Backtracking, BC and BM in the earlier part of the search, but this helps them to prune off search space where no solution could exist.

- As a result, the lookahead algorithms needed less compatibility checking in total in the tested problems.
- Among the lookahead algorithms, Forward Checking does significantly fewer compatibility checks in total than DAC-Lookahead in the problems tested, which in turn does even less checking than AC-Lookahead.

- We must take into consideration the characteristics of the problems being used in the tests.
- The $N$-queens problem is a special CSP, in that it becomes looser as $N$ grows bigger.
- The above results were derived from relatively small problems (problems with 10 variables, 10 values each).
- In realistic applications, the number of variables and domains could be much greater, and the picture needs not be similar.